

PRÁCTICA N.º 1

Introducción a Flutter en Android Studio

Laboratorio resuelto

Campo	Información
Curso	Desarrollo de Software II
Docente	HANS HARLEY CCACYAHUILLCA BEJAR
Estudiante	HECTOR PAOLO BARAZORDA CUELLAR
Integrantes	Completar 2 o 3 integrantes
Fecha	05 de octubre de 2026
Entorno usado	macOS Apple Silicon + Flutter Web + Brave

Nota de reproducibilidad: la guía original solicita Android Studio y un emulador Android. En este equipo se resolvió la práctica utilizando Flutter Web y Brave, que permiten ejecutar y demostrar la aplicación sin instalar un emulador. El código es multiplataforma y también puede abrirse en Android cuando se configure el SDK correspondiente.

1. Objetivo

Comprender la estructura de una aplicación Flutter, diferenciar widgets sin estado y con estado, organizar una interfaz con widgets de layout y crear una aplicación interactiva. Como producto final se implementa una calculadora básica con operaciones aritméticas y retroalimentación visual.

1.1. Entorno verificado

Componente	Estado / versión
Sistema operativo	macOS 26.2, Apple Silicon arm64
Flutter	3.47.5 estable, instalado con Homebrew
Editor	Visual Studio Code con extensiones Dart y Flutter
Navegador	Brave; ejecución mediante web-server
Android Studio	No necesario para esta ejecución web

1.2. Instalación y ejecución

Flutter ya incluye el SDK de Dart. Los siguientes comandos configuran el destino web y ejecutan el proyecto en un servidor local que se abre desde Brave:

Terminal

```
# Instalar Flutter en macOS con Homebrew
brew install --cask flutter

# Verificar el entorno
flutter --version
flutter doctor

# Habilitar la plataforma web
flutter config --enable-web

# Crear el proyecto
flutter create mi_primera_app
cd mi_primera_app

# Ejecutar en un navegador compatible, como Brave
flutter run -d web-server
```

La terminal imprime una dirección local, por ejemplo `http://localhost:8080`. Esa dirección se copia en Brave. Los avisos sobre Android Studio o Xcode pueden permanecer mientras el objetivo sea Web.

1.3. Estructura del proyecto

Archivo / carpeta	Responsabilidad
lib/main.dart	Punto de entrada y código principal de la interfaz.
pubspec.yaml	Nombre, versión, SDK, dependencias y assets.
test/	Pruebas unitarias y de widgets.
web/	Archivos de arranque para el destino web.
android/ e ios/	Código nativo generado para esas plataformas.

2. Ejercicio 1 — Primera aplicación

Se reemplaza el contenido de `lib/main.dart` por una aplicación mínima. `MaterialApp` configura el tema y `Scaffold` proporciona la estructura de la pantalla.

lib/main.dart — solución

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MiApp());
}

class MiApp extends StatelessWidget {
  const MiApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        useMaterial3: true,
      ),
      home: const PantallaInicio(),
    );
  }
}

class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Practica Flutter')),
      body: const Center(
        child: Text(
          'Hola, Flutter!',
          style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
        ),
      ),
    );
  }
}
```

Resultado esperado

Una ventana o pestaña del navegador con una barra superior y el texto centrado “Hola, Flutter!”.

3. Ejercicio 2 — Tarjeta de perfil

La siguiente pantalla demuestra composición con Card, Padding, Column, Row, CircleAvatar, Divider y ElevatedButton. El widget `_Estadistica` recibe datos por constructor y puede reutilizarse.

PantallaInicio y widget auxiliar

```
class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tarjeta de Perfil'),
        backgroundColor: Colors.blue.shade800,
        foregroundColor: Colors.white,
      ),
      backgroundColor: Colors.grey.shade100,
      body: Center(
        child: Card(
          elevation: 8,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
          ),
          child: Padding(
            padding: const EdgeInsets.all(24),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                CircleAvatar(
                  radius: 60,
                  backgroundColor: Colors.blue.shade200,
                  child: const Icon(Icons.person, size: 70, color: Colors.white),
                ),
                const SizedBox(height: 16),
                const Text('Ada Lovelace', style: TextStyle(
                  fontSize: 24, fontWeight: FontWeight.bold,
                )),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

```

    ),
    Text('Ingenieria Informatica', style: TextStyle(
      fontSize: 16, color: Colors.grey.shade600,
    )),
    const Divider(height: 32),
    const Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        _Estadistica(valor: '120', etiqueta: 'Proyectos'),
        _Estadistica(valor: '4.8', etiqueta: 'Rating'),
        _Estadistica(valor: '5+', etiqueta: 'Anos'),
      ],
    ),
    const SizedBox(height: 20),
    ElevatedButton.icon(
      onPressed: null,
      icon: const Icon(Icons.message),
      label: const Text('Enviar Mensaje'),
    ),
  ],
),
),
),
),
);
}
}

class _Estadistica extends StatelessWidget {
  final String valor;
  final String etiqueta;

  const _Estadistica({required this.valor, required this.etiqueta});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(valor, style: const TextStyle(
        fontSize: 22, fontWeight: FontWeight.bold, color: Colors.blue,
      )),
      Text(etiqueta, style: TextStyle(color: Colors.grey.shade600)),
    ]);
  }
}

```

4. Ejercicio 3 — Contador interactivo

Aquí se usa `StatefulWidget` porque el contador cambia durante la ejecución. Cada llamada a `setState` marca el estado como modificable y solicita a Flutter reconstruir la parte afectada del árbol de widgets.

lib/main.dart — widget con estado

```

class PantallaInicio extends StatefulWidget {
  const PantallaInicio({super.key});

  @override
  State<PantallaInicio> createState() => _PantallaInicioState();
}

class _PantallaInicioState extends State<PantallaInicio> {
  int _contador = 0;

  void _incrementar() => setState(() => _contador++);

  void _decrementar() => setState(() {
    if (_contador > 0) _contador--;
  });

  void _reiniciar() => setState(() => _contador = 0);

  @override
  Widget build(BuildContext context) {
    final color = _contador == 0
      ? Colors.grey
      : (_contador < 5 ? Colors.blue : Colors.green);

    return Scaffold(
      appBar: AppBar(title: const Text('Contador Interactivo')),
      body: Center(child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Text('Valor actual:', style: TextStyle(fontSize: 20)),

```

```

Text('$_contador', style: TextStyle(
  fontSize: 80, fontWeight: FontWeight.bold, color: color,
)),
Row(mainAxisAlignment: MainAxisAlignment.center, children: [
  FloatingActionButton(
    heroTag: 'dec', onPressed: _decrementar,
    child: const Icon(Icons.remove),
  ),
  const SizedBox(width: 20),
  FloatingActionButton.extended(
    heroTag: 'rst', onPressed: _reiniciar,
    label: const Text('Reset'), icon: const Icon(Icons.refresh),
  ),
  const SizedBox(width: 20),
  FloatingActionButton(
    heroTag: 'inc', onPressed: _incrementar,
    child: const Icon(Icons.add),
  ),
]),
],
));
}
}

```

5. Cuestionario resuelto

5.1. Diferencia entre StatelessWidget y StatefulWidget

StatelessWidget: representa una vista cuyo resultado depende de propiedades inmutables y no mantiene un estado interno que cambie durante su vida. Ejemplo: un título, una tarjeta de información o un botón configurado por parámetros.

StatefulWidget: separa la configuración del widget de un objeto State que conserva datos mutables. Cuando esos datos cambian se llama setState. Ejemplo: un contador, un formulario o una pantalla cuyo contenido responde a la interacción del usuario.

5.2. ¿Por qué Flutter no usa componentes nativos para todo el renderizado?

Flutter dibuja gran parte de la interfaz con su propio motor gráfico. Así controla de forma consistente la apariencia, la composición y las animaciones entre plataformas, evitando que una misma pantalla dependa demasiado de las diferencias visuales entre Android, iOS y Web. Esto no significa que ignore al sistema operativo: utiliza integraciones nativas para teclado, accesibilidad, ventanas, plugins y otros servicios de plataforma.

5.3. ¿Qué hace flutter doctor?

flutter doctor inspecciona el entorno y reporta qué partes están listas y cuáles requieren configuración. No instala automáticamente todo, pero indica la acción necesaria.

Sección	Qué verifica
Flutter	SDK, canal, versión y componentes internos.
Android toolchain	SDK, licencias, herramientas y Java para Android.
Android Studio	Instalación del IDE y plugins de Flutter/Dart.
Connected device	Emuladores, teléfonos conectados y destinos Web/Desktop.
Network resources	Acceso a servidores necesarios para paquetes y herramientas.

5.4. ¿Qué sucede al llamar setState?

setState ejecuta la función que modifica los datos y notifica a Flutter que el objeto State quedó inválido para la representación actual. Flutter programa un nuevo frame, vuelve a ejecutar build para ese subárbol y compara la descripción de widgets con el árbol existente. Luego actualiza los elementos necesarios y realiza layout, pintura y composición. setState no dibuja directamente ni debe usarse para cambiar datos que no pertenecen a ese State.

5.5. Hot Reload frente a Hot Restart

Aspecto	Hot Reload	Hot Restart
Estado	Conserva el estado actual cuando es posible.	Reinicia el estado de la aplicación.
Velocidad	Más rápido para cambios de UI.	Más lento porque vuelve a ejecutar main().
Uso	Ajustar widgets, estilos y textos.	Probar el arranque desde cero o cambios globales.

6. Ejercicio propuesto — Calculadora básica

La solución usa `StatefulWidget`, un `GridView` de cuatro columnas y un evaluador pequeño para las cuatro operaciones. El color del resultado se obtiene de su signo: azul para positivo, rojo para negativo y gris para cero o cuando todavía no hay resultado.

lib/main.dart — solución completa de la calculadora

```
import 'package:flutter/material.dart';

void main() => runApp(const CalculadoraApp());

class CalculadoraApp extends StatelessWidget {
  const CalculadoraApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.indigo),
      home: const CalculadoraPage(),
    );
  }
}

class CalculadoraPage extends StatefulWidget {
  const CalculadoraPage({super.key});

  @override
  State<CalculadoraPage> createState() => _CalculadoraPageState();
}

class _CalculadoraPageState extends State<CalculadoraPage> {
  String _expression = '';
  String _result = '0';

  bool get _hasOperator => RegExp(r'[+\\-x÷]').hasMatch(_expression);

  Color get _resultColor {
    final value = double.tryParse(_result);
    if (value == null || value == 0) return Colors.grey;
    return value > 0 ? Colors.blue : Colors.red;
  }

  void _clear() => setState(() {
    _expression = '';
    _result = '0';
  });

  void _press(String value) {
    if (value == 'C') return _clear();
    if (value == '=') return _calculate();

    setState(() {
      if (value == '.') {
        final current = _expression.split(RegExp(r'[+\\-x÷]')).last;
        if (current.contains('.')) return;
        if (current.isEmpty) _expression += '0';
      }

      if (RegExp(r'[+\\-x÷]').hasMatch(value)) {
        if (_expression.isEmpty) return;
        if (RegExp(r'[+\\-x÷]$').hasMatch(_expression)) {
          _expression = _expression.substring(0, _expression.length - 1);
        }
        _expression += value;
      }
    });
  }

  void _calculate() {
    if (_expression.isEmpty || !_hasOperator) return;
    try {
      final value = _evaluate(_expression);
      setState(() => _result = _format(value));
    } catch (_) {
      setState(() => _result = 'Error');
    }
  }

  double _evaluate(String expression) {
    final tokens = RegExp(r'\\d+(?:\\.\\d+)?|([+\\-x÷])')

```

```

        .allMatches(expression)
        .map((match) => match.group(0)!)
        .toList();
    if (tokens.length < 3 || tokens.length.isEven) throw FormatException();

    var total = 0.0;
    var term = double.parse(tokens.first);
    for (var i = 1; i < tokens.length; i += 2) {
      final operation = tokens[i];
      final next = double.parse(tokens[i + 1]);
      if (operation == 'x') {
        term *= next;
      } else if (operation == '÷') {
        if (next == 0) throw IntegerDivisionByZeroException();
        term /= next;
      } else {
        total += term;
        term = operation == '+' ? next : -next;
      }
    }
    return total + term;
  }

  String _format(double value) {
    if (value == value.roundToDouble()) return value.toInt().toString();
    return value.toStringAsFixed(6).replaceFirst(RegExp(r'0+$'), '');
  }

  @override
  Widget build(BuildContext context) {
    const buttons = [
      '7', '8', '9', '÷',
      '4', '5', '6', 'x',
      '1', '2', '3', '-',
      'C', '0', '.', '+',
    ];

    return Scaffold(
      appBar: AppBar(title: const Text('Calculadora Flutter')),
      body: Padding(
        padding: const EdgeInsets.all(18),
        child: Column(children: [
          Expanded(
            flex: 2,
            child: Align(
              alignment: Alignment.bottomRight,
              child: Column(mainAxisAlignment: MainAxisAlignment.end,
                crossAxisAlignment: CrossAxisAlignment.end, children: [
                Text(_expression, style: const TextStyle(fontSize: 28)),
                Text(_result, style: TextStyle(
                  fontSize: 54, fontWeight: FontWeight.bold, color: _resultColor,
                )),
              ]),
            ),
          ),
          const SizedBox(height: 16),
          Expanded(
            flex: 4,
            child: GridView.count(
              crossAxisCount: 4,
              crossAxisSpacing: 10,
              mainAxisSpacing: 10,
              children: [
                ...buttons.map((button) => _CalculatorButton(
                  label: button, onPressed: () => _press(button),
                )),
                _CalculatorButton(label: '=', onPressed: _calculate),
              ],
            ),
          ),
        ]),
      );
  }

  class _CalculatorButton extends StatelessWidget {
    final String label;
    final VoidCallback onPressed;

    const _CalculatorButton({required this.label, required this.onPressed});

    @override
    Widget build(BuildContext context) {
      return FilledButton(

```



```
        onPressed: onPressed,  
        child: Text(label, style: const TextStyle(fontSize: 22)),  
      );  
    }  
  }
```

Pruebas manuales

- $2 + 3 = 5$: el resultado aparece en azul.
- $5 - 8 = -3$: el resultado aparece en rojo.
- $4 - 4 = 0$: el resultado aparece en gris.
- $6 \times 7 = 42$ y $8 \div 2 = 4$: se comprueba la prioridad de multiplicación y división.
- C limpia la expresión y devuelve el display a cero.

7. Mostrar la aplicación en un celular

Android Studio no es necesario si solamente se quiere mostrar la versión Web en el navegador del celular. La computadora y el teléfono deben estar conectados a la misma red Wi-Fi. El servidor debe escuchar en todas las interfaces de red, no solo en localhost:

Mostrar en Android o iPhone mediante Brave/Safari

```
# Ejecutar desde la carpeta del proyecto Flutter
flutter run -d web-server --web-hostname 0.0.0.0 --web-port 8080

# En otra terminal, obtener la IP de la Mac conectada por Wi-Fi
ipconfig getifaddr en0

# En el celular abrir, reemplazando la IP:
# http://192.168.1.25:8080
```

- Si macOS pregunta si permite conexiones entrantes, seleccionar Permitir.
- El celular debe usar la misma red Wi-Fi y no la red móvil.
- Esta modalidad muestra la app Web; no instala un APK y la depuración es limitada.

7.1. Ejecutar como aplicación Android nativa

Si la exposición exige una aplicación instalada como APK, sí se necesita configurar el Android SDK. Android Studio no es estrictamente el único camino, pero es el más simple porque instala el SDK, Platform-Tools y las herramientas de compilación desde SDK Manager.

Android físico

```
# Instalar Android Studio en macOS
brew install --cask android-studio

# Después de instalar el SDK desde Android Studio:
flutter doctor --android-licenses
flutter doctor

# En el celular Android activar Opciones de desarrollador y Depuración USB.
# Conectar el teléfono, autorizar la computadora y verificarlo:
flutter devices

# Ejecutar en el teléfono detectado:
flutter run -d <id-del-dispositivo>

# Opcional: generar un APK instalable
flutter build apk --release
```

La documentación oficial de Flutter indica que para Android se debe configurar el SDK, aceptar las licencias y habilitar la depuración USB en el dispositivo físico. Para iPhone se requiere Xcode y la configuración de desarrollo iOS en macOS.

8. Verificación y entrega

8.1. Comandos de verificación

Terminal

```
# Desde la carpeta del proyecto Flutter
flutter analyze
flutter run -d web-server

# Opcional: ejecutar pruebas de widgets
flutter test
```

Para documentar la evidencia, se recomienda capturar: (1) la salida de flutter doctor, (2) la pantalla de la tarjeta de perfil, (3) el contador después de incrementar y reiniciar, y (4) la calculadora con resultados positivo, negativo y cero. La captura debe mostrar la dirección local o el título de la app.

8.2. Conclusiones

Flutter permite construir interfaces declarativas mediante composición de widgets. StatelessWidget sirve para vistas derivadas de datos inmutables, mientras que StatefulWidget permite reaccionar a eventos y actualizar el árbol mediante setState. La calculadora integra estos conceptos en una pantalla funcional y reutilizable. La ejecución Web con Brave reduce la instalación inicial y mantiene el código listo para una futura ejecución Android.

Fuentes

Flutter. (2026). *Flutter documentation: Get started*. <https://docs.flutter.dev/get-started/install>

Dart. (2026). *Dart language tour*. <https://dart.dev/language>

Material Design. (2026). *Flutter Material library*. <https://docs.flutter.dev/ui>